DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University

Submitted by:

Nakul Arora

(2401010025)

Submitted to

Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

SESSION – 2025-2026

Experiment : 1

Question 1. Consider following databases and draw ER diagram and convert entities

and relationships to relation table for a given scenario:

COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Solution:

1. The entities **STUDENT, SEMSEC, SUBJECT, and IAMARKS** are identified and represented using rectangles.
2. The attributes are represented using ellipses:

o **STUDENT:** USN (Primary Key), SName, Address, Phone, Gender
♣ USN is underlined as it is the Primary Key.
♣ Phone is shown as a single-valued attribute.

o **SEMSEC:** SSID (Primary Key), Sem, Sec
♣ SSID is underlined as the Primary Key.

o **SUBJECT:** Subcode (Primary Key), Title, Sem, Credits
♣ Subcode is underlined as the Primary Key.

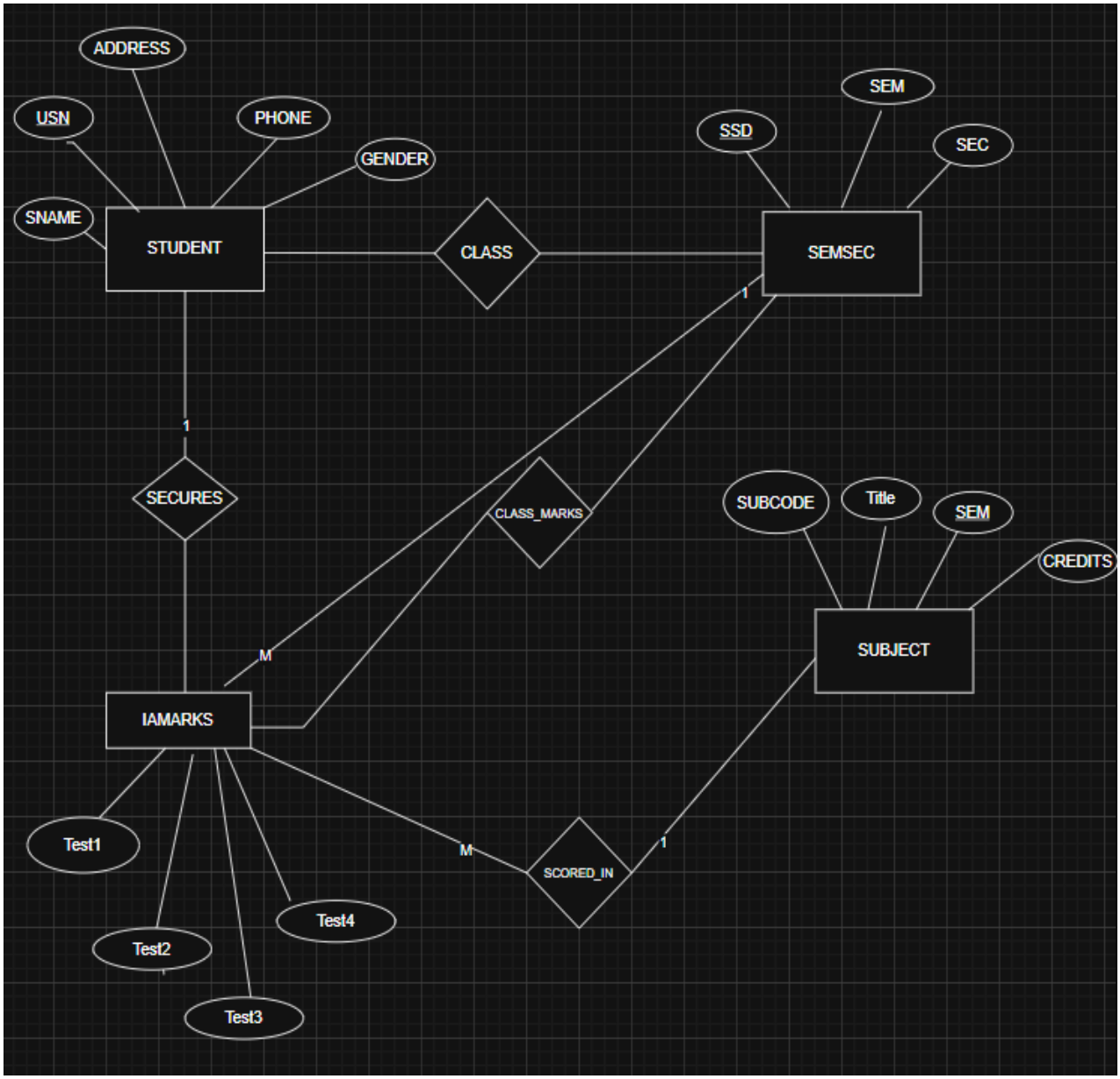
o **IAMARKS:** USN, Subcode, SSID, Test1, Test2, Test3, FinalIA
♣ USN, Subcode, and SSID together form a composite key.
♣ Test1, Test2, Test3, and FinalIA are normal attributes.

3. Primary keys are represented by underlining the attribute names.
4. **CLASS** is drawn as a relationship (diamond shape) between **STUDENT** and **SEMSEC**.
5. The relationship between **STUDENT and SEMSEC** through CLASS is **M : 1**, meaning many students belong to one semester and section.
6. The **M : N relationship between STUDENT and SUBJECT** is resolved using the associative entity **IAMARKS**.
7. The relationship **STUDENT – IAMARKS** is **1 : M**, indicating one student can have multiple internal assessment records.
8. The relationship **SUBJECT – IAMARKS** is **1 : M**, indicating one subject can have multiple IA records for different students.

9. The relationship **SEMSEC – IAMARKS** is **1 : M**, showing that marks are maintained semester-section wise.

10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



Experiment : 2

Question 2. Consider following databases and draw ER diagram and convert entities

and relationships to relation table for a given scenario:

COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Salary, Sex, SuperSSN, DNO)
DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)
PROJECT (PNo, PName, PLocation, DNO)
DLOCATION (DNo, DLoc)
WORKS_ON (SSN, PNo, Hours)
MANAGES (SSN, DNo)

Solution:

1. The entities **EMPLOYEE, DEPARTMENT, PROJECT, and DLOCATION** are identified and represented using rectangles.
2. The attributes are represented using ellipses:

o **EMPLOYEE:** SSN (Primary Key), Name, Address, Salary, Sex, SuperSSN, DNO
♣ SSN is underlined as it is the Primary Key.
♣ SuperSSN is a foreign key referencing EMPLOYEE (self-relationship).
♣ DNO is a foreign key referencing DEPARTMENT.

o **DEPARTMENT:** DNo (Primary Key), DName, MgrSSN, MgrStartDate
♣ DNo is underlined as the Primary Key.
♣ MgrSSN is a foreign key referencing EMPLOYEE.

o **PROJECT:** PNo (Primary Key), PName, PLocation, DNO
♣ PNo is underlined as the Primary Key.
♣ DNO is a foreign key referencing DEPARTMENT.

o **DLOCATION:** DNo, DLoc
♣ DNo and DLoc together form a composite key.
♣ DNo is a foreign key referencing DEPARTMENT.

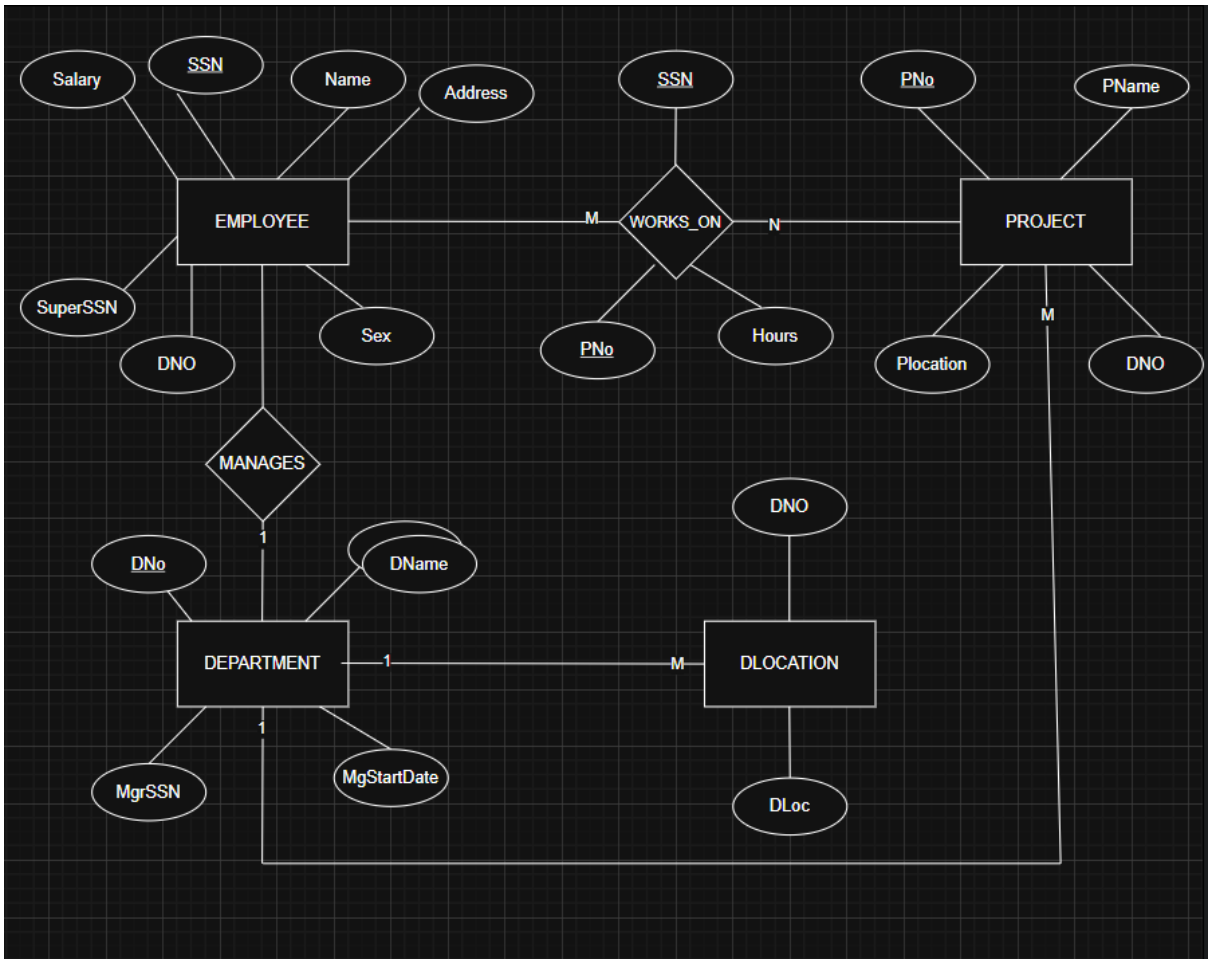
o **WORKS_ON:** SSN, PNo, Hours
♣ SSN and PNo together form a composite key.
♣ Hours is a descriptive attribute of the relationship.

3. Primary keys are represented by underlining the attribute names.
4. The relationship **WORKS_ON** is drawn as a diamond between **EMPLOYEE** and **PROJECT** with **M : N cardinality**, indicating that:
    - One employee can work on many projects.
    - One project can have many employees.

- Hours attribute represents the number of hours worked.

5. The relationship **MANAGES** is drawn between **EMPLOYEE** and **DEPARTMENT** with **1 : 1 cardinality**, indicating:
    - One employee manages one department.
    - Each department has one manager.
6. The relationship between **EMPLOYEE and DEPARTMENT** (via DNO) is **M : 1**, meaning:
    - Many employees work in one department.
7. The relationship between **DEPARTMENT and PROJECT** is **1 : M**, meaning:
    - One department controls many projects.
    - Each project belongs to one department.
8. The relationship between **DEPARTMENT and DLOCATION** is **1 : M**, meaning:
    - One department can have multiple locations.
9. A recursive relationship exists in EMPLOYEE through **SuperSSN**, representing that:
    - One employee can supervise many employees.
10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



Experiment : 3

Question : Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

Output:

```
CREATE DATABASE CUSTOMER_SALE;
USE CUSTOMER_SALE;
CREATE TABLE CUSTOMER(
CUST_ID INT PRIMARY KEY,
CUST_NAME VARCHAR(100)
);
CREATE TABLE ITEM (
ITEM_ID INT PRIMARY KEY,
ITEM_NAME VARCHAR(100),
PRICE INT);
CREATE TABLE SALE (
BILL_NO INT ,
BILL_DATA DATE,
CUST_ID INT,
ITEM_ID INT,
QTY_SOLD INT ,
FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID)
);
```

2. Insert around 10 records in each of the tables.

```
INSERT INTO
CUSTOMER(CUST_ID ,CUST_NAME)
VALUES
(1,'nakul'),
(2,'Harsh'),
(3,'Ankit'),
(4,'Aryan'),
(5,'Hridyanshu'),
(6,'Ashok'),
(7,'Alok'),
(8,'Aman'),
(9,'Ashish'),
(10,'Mohit');
```

CUST_ID	CUST_NAME
1	Nakul
2	Harsh
3	Ankit
4	Aryan
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit
11	RAHUL
NULL	NULL

3.

```sql
INSERT INTO ITEM VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',800),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Watch',2200),
(107,'Mouse',500),
(108,'Keyboard',700),
(109,'Charger',300),
(110,'Headphones',1800);
SELECT * FROM ITEM;
```

ITEM_ID	ITEM_NAME	PRICE
101	Pen	50
102	Notebook	120
103	Bag	800
104	Bottle	250
105	Shoes	1500
106	Watch	2200
107	Mouse	500
108	Keyboard	700
109	Charger	300
110	Headphones	1800
NULL	NULL	NULL

```sql
INSERT INTO SALE VALUES
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,101,3),
(3,CURDATE(),2,105,2),
(4,'2026-02-15',3,104,1),
(5,'2026-02-14',4,106,1),
(6,'2026-02-13',5,102,4),
(7,CURDATE(),6,110,1),
(8,'2026-02-16',7,108,2),
(9,CURDATE(),5,109,5),
(10,'2026-02-12',8,107,1);
SELECT * FROM SALE;
```

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	QTY_SOLD
1	2026-02-23	1	103	1
2	2026-02-23	2	101	3
3	2026-02-23	2	105	2
4	2026-02-15	3	104	1
5	2026-02-14	4	106	1
6	2026-02-13	5	102	4
7	2026-02-23	6	110	1
8	2026-02-16	7	108	2
9	2026-02-23	5	109	5
10	2026-02-12	8	107	1

4. List all the bills for the current date with the customer names and item numbers

```sql
SELECT S.BILL_NO, C.CUST_NAME, S.ITEM_ID
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
WHERE S.BILL_DATA = CURDATE();
```

BILL_NO	CUST_NAME	ITEM_ID
1	Nakul	103
2	Harsh	101
3	Harsh	105
3	Harsh	105
7	Ashok	110
9	Aman	110

5. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
    S.BILL_NO,
    C.CUST_NAME,
    I.ITEM_NAME,
    S.QTY_SOLD,
    I.PRICE,
    (S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
```

BILL_NO	CUST_NAME	ITEM_NAME	QTY_SOLD	PRICE	FINAL_AMC
1	Nakul	Bag	1	800	800
2	Harsh	Pen	3	101	150
3	Harsh	Shoes	2	1500	3000
4	Ankit	Bottle	1	250	250
5	Aryan	Watch	1	2200	2200
6	Hridyanshu	Notebook	4	120	480
7	Ashok	Headphones	1	1800	1800
8	Alok	Keyboard	2	700	1400
9	Hridyanshu	Charger	5	300	1500
10	Aman	Mouse	1	500	500

6. List the details of the customer who have bought a product which has a price>200

```
SELECT DISTINCT C.*
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE I.PRICE > 200;
```

CUST_ID	CUST_NAME
1	Nakul
3	Ankit
2	Harsh
4	Aryan
8	Aman
7	Alok
5	Hridyanshu
6	Ashok

7. Give a count of how many products have been bought by each customer

```
SELECT C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS_BOUGHT
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
GROUP BY C.CUST_NAME;
```

CUST_NAME	TOTAL_PRODUCTS_BOUGHT
Nakul	1
Harsh	5
Ankit	1
Aryan	1
Hridyanshu	9
Ashok	1
Alok	2
Aman	1

8. Give a list of products bought by a customer having cust_id as 5.

```
SELECT I.ITEM_NAME, S.QTY_SOLD
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.CUST_ID = 5;
```

ITEM_NAME	QTY_SOLD
Notebook	4
Charger	5

z

9. List the item details which are sold as of today.

```
SELECT I.*
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.BILL_DATA = CURDATE();
```

ITEM_ID	ITEM_NAME	PRICE
103	Bag	800
101	Pen	50
105	Shoes	1500
110	Headphones	1800
109	Charger	300

10. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold,Amount

```
CREATE VIEW Bill_Details AS
SELECT
    S.BILL_NO,
    S.BILL_DATA,
    S.CUST_ID,
    S.ITEM_ID,
    I.PRICE,
    S.QTY_SOLD,
    (S.QTY_SOLD * I.PRICE) AS AMOUNT
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT * FROM BILL_DETAILS;
```

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	PRICE	QTY_SOLD	AMOUNT
1	2026-02-23	1	103	800	1	800
2	2026-02-23	2	101	50	3	150
3	2026-02-23	2	105	1500	2	3000
4	2026-02-15	3	104	250	1	250
5	2026-02-14	4	106	2200	1	2200
6	2026-02-13	5	102	120	4	480
7	2026-02-23	6	110	1800	1	1800
8	2026-02-16	7	108	700	2	1400
9	2026-02-23	5	109	300	5	1500
10	2026-02-12	8	107	500	1	500

11. Create a view which lists the daily sales date wise for the last one week

```
CREATE VIEW WEEKLY_SALES AS
SELECT
    BILL_DATA,
    SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE BILL_DATA >= CURDATE() - INTERVAL 7 DAY
GROUP BY BILL_DATA;

SELECT * FROM WEEKLY_SALES;
```

BILL_DATA	TOTAL_DAILY_SALES
2026-02-23	7250
2026-02-16	1400

Experiment: 4

Question: Consider the following table: Table: CLASS (ID,Name)

```
CREATE DATABASE NAKUL;
USE NAKUL;
CREATE TABLE CLASS(
ID INT,
NAME VARCHAR(100));
INSERT INTO
CLASS(ID, NAME)
VALUES
(1,'Nakul'),
(2,'Harsh'),
(3,'Ankit'),
(4,'Aryan'),
(5,'Hridyanshu'),
(6,'Ashok'),
(7,'Alok'),
(8,'Aman'),
(9,'Ashish'),
(10,'Mohit');
SELECT * FROM CLASS;
```

ID	NAME
1	Nakul
2	Harsh
3	Ankit
4	Aryan
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit

1. SET AUTOCOMMIT = 0; and START TRANSACTION;

```
SET autocommit = 0;
START TRANSACTION;
```

2. INSERT INTO class VALUES (11,'Rahul');

```
INSERT INTO CLASS VALUES (11,'RAHUL');
SELECT * FROM CLASS;
```

ID	NAME
1	Nakul
2	Harsh
3	Ankit
4	Aryan
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit
11	RAHUL
12	CHRIS
13	BRAVO
11	RAHUL

3. UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;

```
26 •  SET SQL_SAFE_UPDATES = 0;
27 •  UPDATE CLASS SET NAME = 'ABHIJEET' WHERE ID = 5;
28 •  SELECT * FROM CLASS;
```

	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL

4. SAVEPOINT A:

```
28 •  SAVEPOINT A;
```

5. INSERT INTO class VALUES (12, ‘Chris’);

```
30 •  INSERT INTO CLASS VALUES (12,'CHRIS');
```

	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS

6. SAVEPOINT B;

```
31 •  SAVEPOINT B;
```

7. INSERT INTO class VALUES (13, ‘Bravo’);

```
32 •  INSERT INTO CLASS VALUES (13,'BRAVO');
33 •  SELECT * FROM CLASS;
```

	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS
	13	BRAVO

8. SAVEPOINT C;

```
34 •    SAVEPOINT C;
```

9. ROLLBACK TO B;

```
36 •    ROLLBACK TO B;
```

	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS

10. ROLLBACK TO A;

```
38 •    ROLLBACK TO A;
39 •    SELECT * FROM CLASS;
```

	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL

11. COMMIT;

```
40 •    COMMIT;
```

Experiment: 4.2

Consider the following table: Table: ACCOUNT_TXN

Acc_ID Holder_Name Balance

101	**Aman**	**5000**
102	**Bhavna**	**7000**
103	**Chirag**	**6000**

```sql
CREATE TABLE ACCOUNT_TXN(
ACC_ID INT,
HOLDER_NAME VARCHAR(100),
BALANCE INT
);
INSERT INTO ACCOUNT_TXN
VALUES
(101,'AMAN',5000),
(102,'BHAVNA',7000),
(103,'CHIRAG',6000);
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	5000
102	BHAVNA	7000
103	CHIRAG	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

1. INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);

```sql
INSERT INTO ACCOUNT_TXN
VALUES
(104,'DIYA',4000);
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	5000
102	BHAVNA	7000
103	CHIRAG	6000
104	DIYA	4000

2. SAVEPOINT S1

```sql
SET autocommit = 0;
START TRANSACTION;
SAVEPOINT S1;
```

3. UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101

```sql
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE+1500 WHERE ACC_ID = 101;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	6500
102	BHAVNA	7000
103	CHIRAG	6000
104	DIYA	4000

4. **UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102**

```
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE - 1000 WHERE ACC_ID = 102;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	8000
102	BHAVNA	6000
103	CHIRAG	6000
104	DIYA	4000

5. **SAVEPOINT S2;**

```
SAVEPOINT S2;
```

6. **DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103**

```
DELETE FROM ACCOUNT_TXN WHERE ACC_ID = 103;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	8000
102	BHAVNA	6000
104	DIYA	4000

7. **INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);**

```
INSERT INTO ACCOUNT_TXN
VALUES
(105,'ESHAN',3000);
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	8000
102	BHAVNA	6000
104	DIYA	4000
105	ESHAN	3000

8. **SAVEPOINT S3**

```
SAVEPOINT S3;
```

9. **UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;**

```
UPDATE ACCOUNT_TXN SET HOLDER_NAME = 'BHAVNA SHARMA' WHERE ACC_ID = 102;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	AMAN	5000
102	BHAVNA SHARMA	7000
103	CHIRAG	6000
104	DIYA	4000

11. ROLLBACK TO S2;

```
ROLLBACK TO S2;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000

12. INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500)

```
INSERT INTO ACCOUNT_TXN
VALUES
(106,'FARAH',4500);
SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

13. SAVEPOINT S4;

```
SAVEPOINT S4;
```

14. UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN

```
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE + 500 WHERE ACC_ID IN (101,104);
SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	7000
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4500
	104	DIYA	4500
	106	FARAH	4500

15. ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

```
ROLLBACK TO S4;
SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

4000

16. COMMIT

```
COMMIT;
```

17. Identify which rows are permanently saved after COMMIT.

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

Assignment 5

DDL QUESTIONS (Data Definition Language)

1. Create table CLASS with appropriate data types

```
INSERT INTO CLASS VALUES
(1, 'Nakul'),
(2, 'Etish'),
(4, 'Yagya');
```

OUTPUT:

	id	name
▶	1	Nakul
	2	Etish
	4	Yagya

2. Create table EMPLOYEES with specified attributes

```sql
CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(50),
    Phone_Number VARCHAR(20),
    Hire_Date DATE,
    Job_Id VARCHAR(20),
    Salary INT,
    Commission_Pct DECIMAL(5,2),
    Manager_Id INT,
    Department_Id INT
```

OUTPUT:

Field	Type	Null	Key	Default	Extra
Employee_Id	int	NO	PRI	NULL	
First_Name	varchar(50)	YES		NULL	
Last_Name	varchar(50)	YES		NULL	
Email	varchar(50)	YES		NULL	
Phone_Number	varchar(20)	YES		NULL	

3.Create table Client_master with specified attributes

```sql
CREATE TABLE Client_master (
    ClientNo VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50),
    City VARCHAR(50),
    Bal_due INT
);
```

OUTPUT:

	Field	Type	Null	Key	Default	Extra
▶	ClientNo	varchar(10)	NO	PRI	NULL	
	Name	varchar(50)	YES		NULL	
	City	varchar(50)	YES		NULL	
	Bal_due	int	YES		NULL	

4.Rename table Client_master to Client12

```
RENAME TABLE Client_master TO Client12;
```

OUTPUT:

	Tables_in_college_db
▶	class
	client12
	employees

5.Display bal_due column heading as "BALANCE"

```
SELECT Bal_due AS BALANCE FROM Client12;
```

OUTPUT:

Result Grid

	BALANCE

DML QUESTIONS (Data Manipulation Language)

Part A: Transactions and Savepoints (CLASS Table)

1. Insert + Commit

```
INSERT INTO CLASS VALUES (5, 'Rahul');
COMMIT;
```

OUTPUT:

id	name
1	Bravo
2	Alex
4	Cheng
5	Rahul
NULL	NULL

2. Update + Savepoint A

```
UPDATE CLASS SET name = 'Abhijeet' WHERE id = 5;
SAVEPOINT A;
```

OUTPUT:

id	name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
NULL	NULL

3. Insert + Savepoint B

```
INSERT INTO CLASS VALUES (6, 'Chris');
SAVEPOINT B;
```

OUTPUT:

id	name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
6	Chris

4. Insert + Savepoint C

```
INSERT INTO CLASS VALUES (7, 'Bravo');
SAVEPOINT C;
```

OUTPUT:

id	name
4	Cheng
5	Abhijeet
6	Chris
7	Bravo
NULL	NULL

5. Rollback to Savepoint B

```
ROLLBACK TO B;
```

OUTPUT:

id	name
1	Bravo
2	Alex
4	Cheng
5	Abhijeet
6	Chris

6. Rollback to Savepoint A

```
ROLLBACK TO A;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
*	NULL	NULL

PART B: Retrieving Records (EMPLOYEES Table)

1.Find out the employee id, names, and salaries of all the employees.

```
SELECT employee_id, first_name, last_name, salary
FROM employees;
```

OUTPUT:

	employee_id	first_name	last_name	salary
▶	101	Steven	King	24000.00
	102	Neena	Kochhar	17000.00
	103	Lex	De Haan	17000.00
	104	Alexander	Hunold	9000.00
	105	Bruce	Ernst	6000.00

2.List out the employees who work under manager 100.

```
SELECT *
FROM employees
WHERE manager_id = 100;
```

OUTPUT:

employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3.Find the names of the employees who have a salary greater than or equal to 4800.

```
SELECT first_name, last_name
FROM employees
WHERE salary >= 4800;
```

OUTPUT:

first_name	last_name
Steven	King
Neena	Kochhar
Lex	De Haan
Alexander	Hunold
Bruce	Ernst

4.List out the employees whose last name is 'AUSTIN'.

```
SELECT *
FROM employees
WHERE last_name = 'Austin';
```

OUTPUT:

employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
106	David	Austin	DAUSTIN	5904234569	2005-06-25	IT_PROG	4800.00	NULL	103	60
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5.Find the names of the employees who work in departments 60, 70, and 80.

```sql
SELECT first_name, last_name
FROM employees
WHERE department_id IN (60,70,80);
```

OUTPUT:

	first_name	last_name
▶	Alexander	Hunold
	Bruce	Ernst
	David	Austin
	Valli	Pataballa
	Diana	Lorentz

6.Display the unique Manager_Id.

```sql
SELECT DISTINCT manager_id
FROM employees;
```

OUTPUT:

	manager_id
▶	NULL
	101
	102
	104
	103

Assignment 6

Updating Records (Client_master Table)

1.Insert five records into the Client_master table.

```
SELECT * FROM Client_master;
```

OUTPUT:

ClientNO	Name	City	State	Bal_due
C121	Rohit Sharma	Mumbai	Maharashtra	3000
C122	Ananya Verma	Delhi	Delhi	6200
C123	Karan Mehta	Pune	Maharashtra	4800
C124	Sneha Iyer	Bangalore	Karnataka	7500
C125	Amit Khanna	Chandigarh	Punjab	5200

2.Find the names of clients whose bal_due > 5000.

```
SELECT Name
FROM Client_master
WHERE Bal_due > 5000;
```

OUTPUT:

Name
Ananya Verma
Sneha Iyer
Amit Khanna

3.Change the bal_due of ClientNO "C123" to Rs. 5100.

```
UPDATE Client_master
SET Bal_due = 5100
WHERE ClientNO = 'C123';
```

OUTPUT:

	ClientNO	Name	City	State	Bal_due
▶	C123	Karan Mehta	Pune	Maharashtra	5100
*	NULL	NULL	NULL	NULL	NULL